IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 10

Testing dan Dependency Injection

Unit Test, UI Test, Koin DI, dan Debugging di KMP

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026

OUTLINE PERTEMUAN

01	**Review Minggu Lalu** Integrasi AI API
02	**Dependency Injection** Konsep DI dan Koin Framework
03	**Unit Testing** kotlin.test, MockK, Turbine
04	**UI Testing** Compose Test dan Test Tags
05	**Debugging** Tools dan Best Practices
06	**Hands-on Practice** 3 Latihan Praktik

CAPAIAN PEMBELAJARAN

CPMK0501

Mahasiswa mampu menerapkan konsep pemrograman untuk pengembangan perangkat lunak.

CPMK0502

Mahasiswa mampu menjelaskan konsep pemrograman untuk pengembangan perangkat lunak.

Setelah pertemuan ini, mahasiswa mampu:

- ✓ Memahami konsep Dependency Injection dan manfaatnya
- ✓ Mengimplementasikan Koin DI di project KMP
- ✓ Menulis unit test dengan kotlin.test dan MockK
- ✓ Melakukan UI testing dengan Compose Test
- ✓ Menggunakan debugging tools secara efektif

REVIEW MINGGU LALU

Pertemuan 9: Integrasi AI API

OpenAI API	Chat completions, API key management
Gemini API	Google AI integration, multimodal
Prompt Engineering	System prompts, few-shot learning
AI Features	Chat assistant, content generation
Error Handling	Rate limits, fallback strategies

🚀 **Hari ini: Kualitas kode dengan DI dan Testing!**

BAGIAN 1

DEPENDENCY INJECTION

Konsep DI dan Koin Framework

APA ITU DEPENDENCY INJECTION?

Dependency Injection adalah design pattern di mana objek menerima dependencies-nya dari luar, bukan membuat sendiri. Ini membuat kode lebih testable, maintainable, dan loosely coupled.

✗ Tanpa DI (Tight Coupling)

```
class NotesViewModel {
  // Membuat dependency sendiri
  private val repo = NoteRepository(
    NoteDatabase()
  )
}
```

✓ Dengan DI (Loose Coupling)

```
class NotesViewModel(
  // Dependency di-inject
  private val repo: NoteRepository
) {
  // ...
}
```

Manfaat DI:

- Testability - Mudah mock dependencies untuk testing
- Flexibility - Ganti implementasi tanpa ubah consumer
- Reusability - Dependency bisa di-share antar komponen
- Maintainability - Single source of truth untuk object creation

KOIN: DI FRAMEWORK UNTUK KOTLIN

Koin adalah lightweight DI framework untuk Kotlin. Tidak menggunakan code generation atau reflection - murni Kotlin DSL. Sangat cocok untuk Kotlin Multiplatform.

🎯	**Pure Kotlin DSL**	Definisikan modules dengan Kotlin syntax
⚡	**No Reflection**	Fast startup, no hidden magic
📦	**KMP Support**	Berjalan di Android, iOS, Desktop, Web
🧪	**Testing Ready**	Built-in testing utilities

build.gradle.kts

```
commonMain.dependencies {
    implementation("io.insert-koin:koin-core:3.5.3")
    implementation("io.insert-koin:koin-compose:1.1.2")  // For Compose
}
```

KOIN MODULES

di/AppModule.kt

```
val appModule = module {
    // Singleton - satu instance untuk seluruh app
    single<NoteDatabase> { NoteDatabase(get()) }

    // Single dengan interface
    single<NoteRepository> { NoteRepositoryImpl(get()) }

    // Factory - instance baru setiap dipanggil
    factory { NoteValidator() }

    // ViewModel dengan parameter
    viewModel { NotesViewModel(get()) }
    viewModel { params -> NoteDetailViewModel(get(), params.get()) }
}

val networkModule = module {
    single { HttpClient(CIO) { /* config */ } }
    single<ApiService> { ApiServiceImpl(get()) }
}

// Combine modules
val allModules = listOf(appModule, networkModule)
```

KOIN DEFINITION TYPES

`single { }`

Singleton - satu instance, di-share seluruh app

```
single { Database() }
```

`factory { }`

Factory - instance baru setiap request

```
factory { Validator() }
```

`viewModel { }`

ViewModel scope - tied to lifecycle

```
viewModel { MyViewModel(get()) }
```

`scope { }`

Custom scope - manual lifecycle control

```
scope<Session> { scoped { UserData() } }
```

CARA MENGGUNAKAN KOIN

1. Start Koin

```
// commonMain - App.kt
fun initKoin() {
    startKoin {
        modules(allModules)
    }
}

// Android - MainActivity.kt
class MainActivity : ComponentActivity() {
    override fun onCreate(...) {
        initKoin()
    }
}
```

2. Inject Dependencies

```
// Constructor injection (recommended)
class NotesViewModel(
    private val repo: NoteRepository
) : ViewModel()

// Lazy injection in Composable
@Composable
fun NotesScreen() {
    val vm: NotesViewModel = koinViewModel()
    // ...
}
```

3. Injection Methods

```
// get() - dalam module definition
single { NoteRepository(get()) }  // get() mengambil NoteDatabase

// koinInject() - dalam Composable
val repo: NoteRepository = koinInject()

// by inject() - lazy delegation di class
class MyClass : KoinComponent {
    private val repo: NoteRepository by inject()
}
```

KOIN BEST PRACTICES

✓ **Use interfaces**

Definisikan dengan interface untuk testability: single<Repository> { RepoImpl() }

✓ **Organize modules**

Pisahkan modules berdasarkan feature/layer: dataModule, domainModule, uiModule

✓ **Constructor injection**

Prefer constructor injection over property injection untuk dependencies yang wajib

✓ **Verify modules**

Gunakan checkModules() di unit test untuk validasi dependency graph

✗ **Avoid Service Locator**

Jangan panggil get() langsung di business logic - inject via constructor

✗ **Don't overuse single**

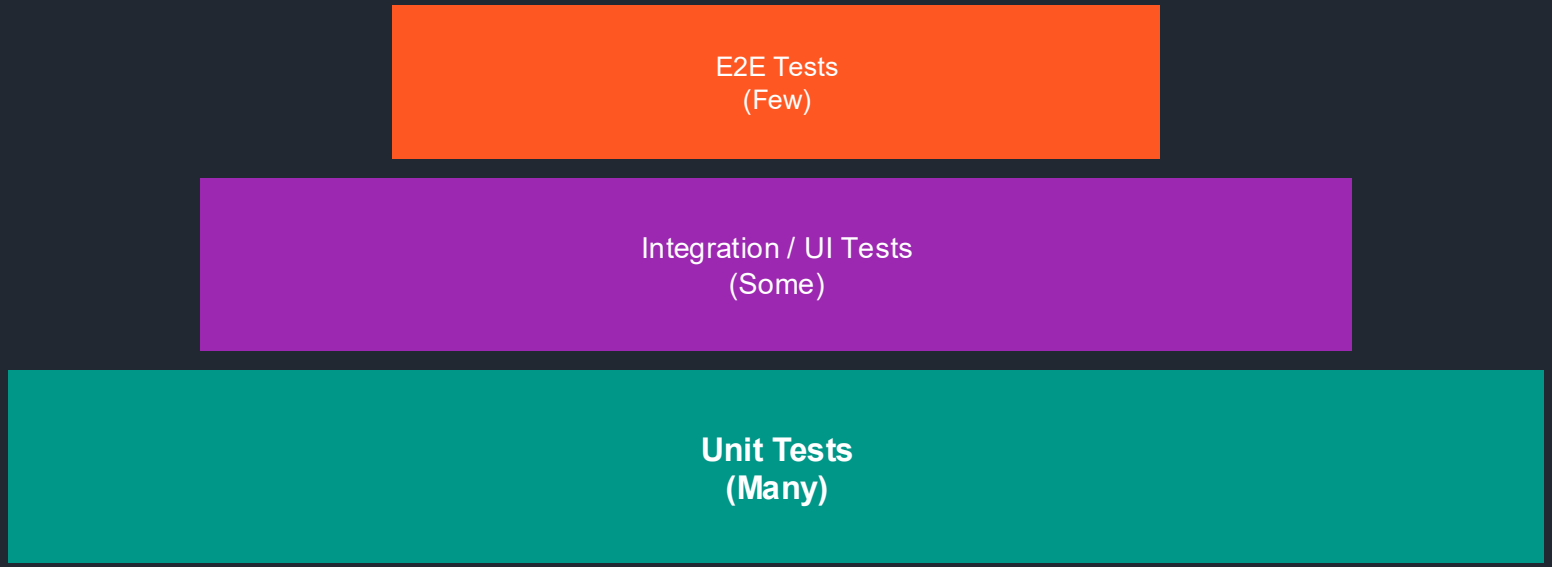
Tidak semua class perlu singleton - gunakan factory untuk stateless objects

BAGIAN 2

UNIT TESTING

kotlin.test, MockK, dan Turbine

TESTING PYRAMID



Karakteristik Testing di KMP:

- commonTest - Test yang berjalan di semua platform (majority)
- androidUnitTest - Android-specific tests
- iosTest - iOS-specific tests

KOTLIN.TEST SETUP

build.gradle.kts

```
kotlin {
    sourceSets {
        commonTest.dependencies {
            implementation(kotlin("test"))  // kotlin.test assertions
            implementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.7.3")
            implementation("app.cash.turbine:turbine:1.0.0")  // Flow testing
            implementation("io.mockk:mockk:1.13.9")  // Mocking (JVM only)
            implementation("io.insert-koin:koin-test:3.5.3")  // Koin testing
        }
    }
}
```

Test Source Sets Structure:

```
src/
├── commonMain/kotlin/        # Shared code
├── commonTest/kotlin/        # Shared tests ← Focus di sini!
│   └── NoteRepositoryTest.kt
│   └── NotesViewModelTest.kt
├── androidUnitTest/kotlin/   # Android-specific tests
└── iosTest/kotlin/           # iOS-specific tests
```

BASIC UNIT TEST

NoteValidatorTest.kt

```
class NoteValidatorTest {
    private lateinit var validator: NoteValidator

    @BeforeTest
    fun setup() {
        validator = NoteValidator()
    }

    @Test
    fun `valid note returns true`() {
        // Arrange
        val note = Note(title = "Shopping", content = "Buy milk")

        // Act
        val result = validator.isValid(note)

        // Assert
        assertTrue(result)
    }

    @Test
    fun `empty title returns false`() {
        val note = Note(title = "", content = "Content")
        assertFalse(validator.isValid(note))
    }

    @Test
    fun `title too long throws exception`() {
        val longTitle = "a".repeat(300)
        assertFailsWith<ValidationException> {
            validator.validate(Note(title = longTitle, content = ""))
        }
    }
}
```

KOTLIN.TEST ASSERTIONS

`assertEquals(expected, actual)`	Nilai harus sama
`assertNotEquals(illegal, actual)`	Nilai harus berbeda
`assertTrue(condition)`	Kondisi harus true
`assertFalse(condition)`	Kondisi harus false
`assertNull(value)`	Nilai harus null
`assertNotNull(value)`	Nilai tidak boleh null
`assertIs<Type>(value)`	Nilai harus bertipe Type
`assertContains(collection, element)`	Collection berisi element
`assertFailsWith<Exception> { }`	Block harus throw exception

MOCKK: MOCKING LIBRARY

MockK adalah mocking library untuk Kotlin. Berguna untuk mengisolasi unit yang ditest dari dependencies-nya.

MockK Basics

```
class NotesViewModelTest {
    // Create mocks
    private val mockRepository = mockk<NoteRepository>()
    private lateinit var viewModel: NotesViewModel

    @BeforeTest
    fun setup() {
        // Stub method returns
        coEvery { mockRepository.getAllNotes() } returns flowOf(listOf(testNote))

        // Create SUT with mock
        viewModel = NotesViewModel(mockRepository)
    }

    @Test
    fun `deleteNote calls repository delete`() = runTest {
        // Stub void method
        coEvery { mockRepository.deleteNote(any()) } just Runs

        viewModel.deleteNote(1L)

        // Verify interaction
        coVerify { mockRepository.deleteNote(1L) }
    }
}
```

MOCKK FUNCTIONS

Creating Mocks

```
// Basic mock
val mock = mockk<MyClass>()

// Relaxed mock (returns defaults)
val relaxed = mockk<MyClass>(relaxed = true)

// Spy (wrap real object)
val spy = spyk(RealClass())

// Mock with annotation
@MockK lateinit var mock: MyClass
```

Stubbing

```
// Regular function
every { mock.getData() } returns "data"

// Suspend function
coEvery { mock.fetchData() } returns data

// Throw exception
every { mock.risky() } throws Exception()

// Void function
every { mock.doSomething() } just Runs
```

Verification

```
// Verify called
verify { mock.method() }
coVerify { mock.suspendMethod() }  // For suspend

// Verify call count
verify(exactly = 2) { mock.method() }
verify(atLeast = 1) { mock.method() }

// Verify not called
verify(exactly = 0) { mock.method() }
```

TURBINE: FLOW TESTING

Turbine adalah library untuk testing Kotlin Flow. Memudahkan testing emissions, errors, dan completion.

Flow Testing

```
@Test
fun `test flow emissions`() = runTest {
    val repository = NoteRepositoryImpl(database)

    repository.getAllNotes().test {
        // Await and assert first emission
        val loading = awaitItem()
        assertEquals(emptyList<Note>(), loading)

        // Add a note
        repository.insertNote(testNote)

        // Await next emission
        val updated = awaitItem()
        assertEquals(1, updated.size)
        assertEquals("Test", updated[0].title)

        // Cancel and ignore remaining
        cancelAndIgnoreRemainingEvents()
    }
}

// Other Turbine functions:
// awaitComplete() - Wait for flow completion
// awaitError() - Wait for error
// expectNoEvents() - Assert nothing emitted
// skipItems(n) - Skip n emissions
```

TESTING VIEWMODEL

```
class NotesViewModelTest {
    private val mockRepo = mockk<NoteRepository>()
    private lateinit var viewModel: NotesViewModel

    @BeforeTest
    fun setup() {
        coEvery { mockRepo.getAllNotes() } returns flowOf(listOf(testNote))
        viewModel = NotesViewModel(mockRepo)
    }

    @Test
    fun `initial state is loading then success`() = runTest {
        viewModel.uiState.test {
            // Initial loading state
            val loading = awaitItem()
            assertIs<NotesUiState.Loading>(loading)

            // Success with data
            val success = awaitItem()
            assertIs<NotesUiState.Success>(success)
            assertEquals(1, success.notes.size)

            cancelAndIgnoreRemainingEvents()
        }
    }

    @Test
    fun `addNote calls repository and refreshes`() = runTest {
        coEvery { mockRepo.insertNote(any()) } just Runs

        viewModel.addNote("New Note", "Content")

        coVerify { mockRepo.insertNote(match { it.title == "New Note" }) }
    }
}
```

TESTING DENGAN KOIN

```
class KoinModuleTest : KoinTest {

    @Test
    fun `check all modules`() {
        // Verify all dependencies can be resolved
        koinApplication {
            modules(allModules)
            checkModules()
        }
    }
}

class NotesViewModelKoinTest : KoinTest {

    @BeforeTest
    fun setup() {
        startKoin {
            modules(
                module {
                    // Override with mock
                    single<NoteRepository> { mockk(relaxed = true) }
                    viewModel { NotesViewModel(get()) }
                }
            )
        }
    }

    @AfterTest
    fun tearDown() {
        stopKoin()
    }

    @Test
    fun `viewModel is injected correctly`() {
        val viewModel: NotesViewModel by inject()
        assertNotNull(viewModel)
    }
}
```

BAGIAN 3

UI TESTING

Compose Test dan Test Tags

COMPOSE TEST SETUP

build.gradle.kts

```
kotlin {
    sourceSets {
        val androidInstrumentedTest by getting {
            dependencies {
                implementation("androidx.compose.ui:ui-test-junit4")
                debugImplementation("androidx.compose.ui:ui-test-manifest")
            }
        }
    }
}
```

Basic UI Test

```
class NotesScreenTest {
    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun notesScreen_displaysTitle() {
        composeTestRule.setContent {
            NotesScreen()
        }

        composeTestRule
            .onNodeWithText("My Notes")
            .assertIsDisplayed()
    }
}
```

COMPOSE TEST FINDERS

`onNodeWithText("text")`	Cari node dengan text tertentu
`onNodeWithContentDescription("desc")`	Cari berdasarkan content description
`onNodeWithTag("tag")`	Cari berdasarkan test tag
`onAllNodesWithText("text")`	Cari semua nodes dengan text
`onRoot()`	Root composable
`onNode(hasText("x") and hasClickAction())`	Kombinasi matchers

```
// Example: Find button with specific text and click it
composeTestRule.onNodeWithText("Add Note").performClick()
```

ACTIONS & ASSERTIONS

🎯 Actions

```
performClick()
performTextInput("text")
performTextClearance()
performScrollTo()
performTouchInput {
  swipeUp()
  swipeDown()
  longClick()
}
```

✓ Assertions

```
assertIsDisplayed()
assertIsNotDisplayed()
assertExists()
assertDoesNotExist()
assertIsEnabled()
assertIsNotEnabled()
assertTextEquals("text")
assertHasClickAction()
```

Example

```
@Test
fun addNote_showsInList() {
    composeTestRule.setContent { NotesScreen() }

    // Type note title
    composeTestRule.onNodeWithTag("titleInput").performTextInput("Shopping")
    // Click add button
    composeTestRule.onNodeWithText("Add").performClick()
    // Verify note appears in list
    composeTestRule.onNodeWithText("Shopping").assertIsDisplayed()
}
```

TEST TAGS

Test Tags memungkinkan testing yang tidak bergantung pada text/content. Lebih reliable untuk localization dan UI changes.

Define Tags

```
// TestTags.kt
object TestTags {
    const val NOTES_LIST = "notes_list"
    const val NOTE_ITEM = "note_item"
    const val TITLE_INPUT = "title_input"
    const val ADD_BUTTON = "add_button"
    const val DELETE_BUTTON = "delete_button"
}

// In Composable
TextField(
    modifier = Modifier.testTag(TestTags.TITLE_INPUT),
    ...
)
```

Use in Tests

```
@Test
fun deleteNote_removesFromList() {
    composeTestRule.setContent {
        NotesScreen()
    }

    // Use tags instead of text
    composeTestRule
        .onNodeWithTag(TestTags.NOTE_ITEM)
        .assertIsDisplayed()

    composeTestRule
        .onNodeWithTag(TestTags.DELETE_BUTTON)
        .performClick()

    composeTestRule
        .onNodeWithTag(TestTags.NOTE_ITEM)
        .assertDoesNotExist()
}
```

💡 **Best Practice: Gunakan test tags untuk elemen yang sering ditest. Text-based selectors rentan terhadap perubahan UI.**

BAGIAN 4

DEBUGGING

Tools dan Best Practices

DEBUGGING TOOLS

Debugger	Breakpoints, step through code, inspect variables
Logcat	View logs, filter by tag/level, search
Layout Inspector	Inspect Compose hierarchy, view properties
Profiler	CPU, Memory, Network, Energy profiling
Database Inspector	View dan query database SQLite/Room
Network Inspector	Monitor HTTP requests/responses

LOGGING DENGAN NAPIER

Napier adalah multiplatform logging library. Logs di Android menggunakan Logcat, di iOS menggunakan os_log.

```
// Setup (call once at app start)
Napier.base(DebugAntilog())  // Enable debug logging

// Logging levels
Napier.v("Verbose message")      // Trace/verbose
Napier.d("Debug message")        // Debug
Napier.i("Info message")         // Info
Napier.w("Warning message")      // Warning
Napier.e("Error message")        // Error

// With tag
Napier.d(tag = "NotesVM") { "Loading notes..." }

// With exception
try {
    riskyOperation()
} catch (e: Exception) {
    Napier.e("Operation failed", e)
}

// Lazy logging (only evaluated if level is enabled)
Napier.d { "Expensive operation: ${computeValue()}" }
```

BREAKPOINTS & DEBUGGER

Line Breakpoint	`Click line number`	Stop at specific line
Conditional	`Right-click breakpoint`	Stop only when condition is true
Exception	`Run > View Breakpoints`	Stop when exception thrown
Method	`Ctrl+Shift+F8`	Stop at method entry/exit

Debugger Controls:

F8
Step Over - next line

F7
Step Into - enter method

Shift+F8
Step Out - exit method

F9
Resume - continue execution

BAGIAN 5

HANDS-ON PRACTICE

3 Latihan Praktik

LATIHAN 1: SETUP KOIN DI

Implementasi Dependency Injection dengan Koin

Tasks

```
// 1. Create modules
val dataModule = module {
    single { NoteDatabase() }
    single<NoteRepository> {
        NoteRepositoryImpl(get())
    }
}

val viewModelModule = module {
    viewModel { NotesViewModel(get()) }
}

val allModules = listOf(
    dataModule, viewModelModule
)

// 2. Initialize in App.kt
fun initKoin() {
    startKoin { modules(allModules) }
}
```

Checklist:

- ☐ Add Koin dependencies
- ☐ Create dataModule
- ☐ Create viewModelModule
- ☐ Initialize Koin
- ☐ Inject ViewModel
- ☐ Verify app runs

⏱ Waktu: 20 menit

LATIHAN 2: UNIT TEST VIEWMODEL

Testing ViewModel dengan MockK dan Turbine

Test Cases

```
class NotesViewModelTest {
    private val mockRepo = mockk<NoteRepository>()

    @Test
    fun `initial state emits loading then success`() =
        runTest {
            coEvery { mockRepo.getAllNotes() } returns
                flowOf(listOf(testNote))

            val vm = NotesViewModel(mockRepo)

            vm.uiState.test {
                assertIs<Loading>(awaitItem())
                assertIs<Success>(awaitItem())
                cancelAndIgnoreRemainingEvents()
            }
        }

    @Test
    fun `deleteNote calls repository`() = runTest {
        coEvery { mockRepo.deleteNote(any()) } just Runs
        val vm = NotesViewModel(mockRepo)

        vm.deleteNote(1L)

        coVerify { mockRepo.deleteNote(1L) }
    }
}
```

Checklist:

- ☐ Setup MockK mock
- ☐ Test loading state
- ☐ Test success state
- ☐ Test error state
- ☐ Test addNote
- ☐ Test deleteNote
- ☐ Verify interactions

⏱️ Waktu: 25 menit

LATIHAN 3: UI TEST

Compose UI Testing dengan Test Tags

UI Tests

```
class NotesScreenTest {
    @get:Rule
    val rule = createComposeRule()

    @Test
    fun emptyState_showsMessage() {
        rule.setContent { NotesScreen(emptyList()) }

        rule.onNodeWithTag(TestTags.EMPTY_STATE)
            .assertIsDisplayed()
    }

    @Test
    fun addNote_showsInList() {
        rule.setContent { NotesScreen() }

        rule.onNodeWithTag(TestTags.TITLE_INPUT)
            .performTextInput("New Note")

        rule.onNodeWithTag(TestTags.ADD_BUTTON)
            .performClick()

        rule.onNodeWithText("New Note")
            .assertIsDisplayed()
    }
}
```

Checklist:

- ☐ Add test tags
- ☐ Test empty state
- ☐ Test notes displayed
- ☐ Test add interaction
- ☐ Test delete interaction
- ☐ Test search/filter

⏱️ Waktu: 20 menit

TUGAS PRAKTIKUM MINGGU 10

Bobot: 4% | Deadline: Sebelum Pertemuan 11

📝 Deskripsi Tugas:

Implementasi DI dan Testing untuk Notes App:

1. Setup Koin DI dengan minimal 2 modules (data, viewModel)
2. Unit test untuk NoteRepository (minimal 5 test cases)
3. Unit test untuk NotesViewModel dengan MockK (minimal 4 test cases)
4. Flow test dengan Turbine (minimal 2 test cases)
5. UI test untuk NotesScreen (minimal 3 test cases)
6. Minimum code coverage 60% untuk business logic

📤 Format Pengumpulan:

- Push ke GitHub repository (branch: week-10)
- README: Test coverage report screenshot, daftar test cases
- Video demo (45 detik): menjalankan semua test dan menunjukkan hasil

RUBRIK PENILAIAN TUGAS

Komponen	Bobot	Kriteria
Koin DI Setup	20%	2+ modules, proper injection
Repository Tests	20%	5+ test cases, all passing
ViewModel Tests	20%	MockK, 4+ test cases
Flow Tests	15%	Turbine, 2+ test cases
UI Tests	15%	Compose test, 3+ test cases
Code Quality	10%	Clean code, AAA pattern

🌟 **Bonus (+10%): Coverage > 80%**

⚠️ **Late: -10%/day, Plagiat: 0**

SUMBER PUSTAKA

Docs

Koin Documentation — insert-koin.io/docs

Docs

kotlin.test — kotlinlang.org/api/latest/kotlin.test

GitHub

MockK — mockk.io

GitHub

Turbine — github.com/cashapp/turbine

Docs

Compose Testing — developer.android.com/jetpack/compose/testing

PREVIEW MINGGU DEPAN

Pertemuan 11: Project Sprint 1 - Planning dan Setup

Project Requirements

Define scope dan features

Architecture Design

Clean Architecture, Modules

Repository Setup

GitHub, branching strategy

CI/CD Basic

GitHub Actions, automation

Persiapan:

- Selesaikan tugas Testing & DI
- Brainstorm ide project akhir
- Review semua materi dari pertemuan 1-10

SESI TANYA JAWAB

Ada pertanyaan tentang DI dan Testing?

Terima Kasih

Pertemuan 10: Testing dan Dependency Injection

> *"Testing leads to failure,*
> *and failure leads to understanding."*
>
> - Burt Rutan

Happy Testing! 🧪✨